```
import pandas as pd
```

```
# 1..Q1. Write a python program which mask a Series based on
# conditions (condition: > 5) with the number n (input) and modify the
# masked values:
num=int(input('enter a number:'))
s=pd.Series([10, 5, 8, 12, 4, 3, 7, 1])

print('original series:')
print(s)

s[s>5]=num
print('modifiy series:')
print(s)
```

```
enter a number:-1
original series:
0    10
1     5
2     8
3    12
4     4
5     3
6     7
7     1
dtype: int64
modifiy series:
0   -1
1    5
2   -1
3   -1
4    4
5    3
6   -1
7    1
dtype: int64
```

```
# Q2. Identify missing values in a Series and replace them with the
# median:
import numpy as np
s=pd.Series([2,5,np.nan,3,9,np.nan,4])
print('original series:')
print(s)

median_value=s.median()
print('modify series:')
s=s.fillna(median_value)
print(s)
```

```
original series:
0    2.0
1    5.0
2    NaN
3    3.0
4    9.0
5    NaN
6    4.0
dtype: float64
modify series:
0    2.0
1    5.0
2    4.0
3    3.0
4    9.0
5    4.0
6    4.0
dtype: float64
```

```
# Q3. Map values from another Series based on a condition:
# inpt=([10, 20, 30, 40]) mapping: {10: 'A', 30: 'B'}
# Output: Series(['A', 20, 'B', 40])

import pandas as pd
s=pd.Series([10, 20, 30, 40])
mapping={10: 'A', 30: 'B'}

result=s.replace(mapping)
print('original series:')
print(s)

print('modified series:')
```

```
print(result)
```

```
original series:
0    10
1    20
2    30
3    40
dtype: int64
modified series:
0     A
1    20
2     B
3    40
dtype: object
```

```
# Q4. Calculate cumulative product of values, ignoring zeroes
import pandas as pd
s=pd.Series([1,2,0,4,5])

result=s.replace(0,1).cumprod()
print('origina series:')
print(s)

print('modified series:')
print(result)
```

```
origina series:
0    1
1    2
2    0
3    4
4    5
dtype: int64
modified series:
0     1
1     2
2     2
3     8
4    40
dtype: int64
```

```
# Q5. Write a python program which Shift elements of a Series and fill
# shifted positions with custom values:
import pandas as pd
s=pd.Series([10, 20, 30, 40])

shift=2
fill_value=-1
result=s.shift(shift,fill_value=fill_value)
print('original series:')
print(s)
print('shifted series:')
print(result)
```

```
original series:
0    10
1    20
2    30
3    40
dtype: int64
shifted series:
0    -1
1    -1
2    10
3    20
dtype: int64
```

```
# Q6. Generate random numbers in a Series and count how many are
# greater than the standard deviation:
import pandas as pd
s=pd.Series([1.2,2.5,3.6,4.1,5.9])

std_value=s.std()
result=(s>std_value).sum()
print('original_series:')
print(s)
print('modify series:')
print(result)
```

```
original_series:
0    1.2
1    2.5
2    3.6
3    4.1
```

```
4    5.9
dtype: float64
modify series:
4
```

```
# Q7. Convert negative values in a Series to absolute values:
import pandas as pd
s=pd.Series([-10,5,-3,2,-7])

result=s.abs()
print('original_series:')
print(s)
print('modify_series:')
print(result)
```

```
original_series:
0   -10
1     5
2    -3
3     2
4    -7
dtype: int64
modify_series:
0    10
1     5
2     3
3     2
4     7
dtype: int64
```

```
# Q8. Find positions of prime numbers in a Series and return the count
import pandas as pd

s = pd.Series([10, 7, 3, 15, 17])

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

prime_positions = [i for i, x in enumerate(s) if is_prime(x)]

print("Prime positions:", prime_positions)
print("Count:", len(prime_positions))
```